

README

for app-compatibility artifacts

❖ Contents

Section I: Content of this artifact package	1
Section II: Explore the artifact	2
Section III: Reproduce the artifact (partially)	4
Section IV: Known issues	7
Appendix: use the artifact on a different machine	7

Section I: Content of this artifact package

The artifact folder we shared is named **appCompatAE**, it includes two files: **artifact.zip** and **VBox.zip**. The first includes the artifacts that correspond to results presented in our paper, and it also includes reusable tools/utilities for similar future studies that we intent to share publicly. The second holds a Virtual Box VM that is used for facilitating the use/replication of our artifacts.

The **appCompatAE** folder is located online: <https://eecs.wsu.edu/~hcai/appCompatAE/>

Once the **artifact.zip** file is downloaded, unpack it. Then the following structure should be seen:

- code
 - *.sh (shell scripts)
 - *.py (python scripts)
 - installationLogs (part of our installation logs)
 - runtimeLogs (example run-time app traces and Monkey logs)
 - samples (the folder of four sample apks, for demonstrating the reusability of our utilities)
 - various other pre-generated intermediate results used for demonstration purposes and for speeding up the artifact evaluation if necessary
- data
 - benchmarks (list of apks per year)
 - *.xlsx (raw data underlying the figures / tables presented in the research paper)
 - *.docx (various study documents)
 - *.m (Matlab scripts used for data statistics and visualizations)
- README.PDF, README.DOC (i.e., this document; same content in different formats)
- Issta19-paper170.pdf (the research paper submitted and accepted)

Then, download the **VBox.zip** file, unzip it, the following files can be found therein

- appcompatAE.vbox

- appcompat.vdi

The **code** and **data** folders include the analysis code and our dataset, respectively. Due to their large sizes, we could not include (1) all the APK files themselves, nor (2) all the app execution traces for the run-time compatibility study. Also, since our entire study took over half a year to finish, for this artifact evaluation, it is not feasible to reproduce all the original traces (for run-time compatibility study) either, nor is it feasible to reproduce all the installation logs (for the installation-time compatibility study). However, to facilitate this artifact evaluation and show the reusability of our utilities, we provided all the experimentation scripts, and gave instructions on how to use them against simple samples (see the ‘Partial reproduction’ section below).

The VBox zip includes the VM for VirtualBox that we created to ease the artifact evaluation, especially for exploring our study and reproducing *part of* the results presented in our research paper. Specifically, two files are included in this folder: the *appcompatAE.vbox* file is the VirtualBox configuration/project file, and the *appcompat.vdi* file is the virtual disk image that actually holds the entire VM OS along with our artifacts.

To load the VM (called *appcompatAE*), please first install VirtualBox 5.1.22 from <https://www.virtualbox.org/wiki/Downloads>. Then either open the *appcompatAE.vbox* file with VirtualBox or create a new Linux 64bit VM by using the VDI image *appcompat.vdi* instead of creating a new virtual hard disk. (Please make sure that your machine itself supports VM running a 64bit OS; enabling this support sometimes needs to change BIOS options.)

We have verified the workings of this VM on Ubuntu 16.04 LTS (the VM OS itself is also Ubuntu 16.04 LTS). We set 8GB RAM for this VM during our test. Once installed, launch the VM. If logon is asked (we have set auto logon on the VM), use the following credentials:

Username: **osboxes** (or an alias named **hcai**)

Password: **appcompat** (also as the sudo password)

The next two sections assume that you have launched and logged in to the VM.

Section II: Explore the artifact

First, open the terminal on the VM, then explore as follows (for convenience, you can always use “tree *dirname*” to explore a directory *dirname*’s structure; the *tree* tool has been installed).

Most of the following steps concern only browsing original inputs to the study, intermediate files produced, raw study results, and final results reported in research paper. We only provide instructions to partially reproduce the study (see next Section of this document for partial reproduction) due to the total time cost of the study. Our study needed to install 62,894 individual apps on eight different emulators (each for a different Android version); more critically, it needed to instrument 15,045 apps and run each individual app for five minutes and then to repeat this process for each of the eight emulators. The trace storage space was over 200GB.

1. Check the study tools/utilities (the ‘code’ folder)

The source code can be found under `/home/hcai/issta-ae/code`, where the content is the same as `code/` in `artifact.zip`. For the dynamic analysis part of our study, we used a toolkit we developed before, found at `/home/hcai/workspace/droidfax`. All dependence libraries are located at `/home/hcai/libs`. To rebuild the `droidfax` code:

```
cd /home/hcai/workspace/droidfax
ant clean build
```

Part of the droidfax (common-purpose scripts) can be found in `/home/hcai/bin` for usage convenience. We used droidfax in this study mainly for APK instrumentation that is needed for studying run-time compatibility issues.

The `code/installationLogs` folder contains part of our installation logs, generated by executing `installAndroZooApks.sh` on our benchmarks. Our installation-time compatibility study results came from these logs.

The `code/runtimeLogs` folder contains just a couple of example traces (and associated Monkey logs) for one year and one emulator (i.e., one SDK version). Our run-time compatibility study results came from these traces (and Monkey logs).

2. Check the study dataset (the ‘data’ folder)

The following table provides a mapping between the generated results and figure/table used in the research paper, along with the note on how the figure/table was produced.

Under data	In paper	Note
Failed Installation-iir.xlsx	Figure 2	The figure was plotted from sheet ‘Summary’ which summarizes the other sheets.
stacked-bar.xlsx	Figure 3	From the sheet ‘install’
data-all-IIR.xlsx	Figure 4 (leftmost)	From the script <code>drawdata_all_IIR.m</code> on this data
installforSPSS.xlsx	Figure 4 (middle)	From the script <code>drawdata_IIR_abi.m</code> on this data (sheet ‘abi’)
installforSPSS.xlsx	Figure 4 (rightmost)	From the script <code>drawdata_IIR_lib.m</code> on this data (sheet ‘library’)
Failed Execution-rir.xlsx	Figure 5	The figure was plotted from sheet ‘Summary’ which summarizes the other sheets.
stacked-bar.xlsx	Figure 6	From the sheet ‘run’
run-timeforSPSS.xlsx	Figure 7 (leftmost)	From the script <code>drawdata_all_RIR.m</code> on this data (sheet ‘total (all)’)
run-timeforSPSS.xlsx	Figure 7 (middle)	From the script <code>drawdata_RIR_verify.m</code> on this data (sheet ‘verify’)
run-timeforSPSS.xlsx	Figure 7 (rightmost)	From the script <code>drawdata_RIR_native.m</code> on this data (sheet ‘native’)
Installation Analysis Result.docx	Table 2	Summarized in <code>spearman-correlation.xlsx</code> ; computed using SPSS from <code>installforSPSS.xlsx</code>
Run-Time Analysis	Table 3	Summarized in <code>spearman-correlation.xlsx</code> ;

Result.docx		computed using SPSS from run-timeforSPSS.xlsx
-------------	--	---

This folder also includes two additional Word documents, “[Explanations and reasons of installation failure effects.docx](#)” and “[Explanations and reasons of Run-time failure effects.docx](#)”, describing the source information we used for understanding/explaining the symptoms of installation- and run-time compatibility issues, respectively.

Section III: Reproduce the artifact (partially)

As explained above, fully reproducing all the results presented in the research paper would not be feasible with respect to the time for this artifact evaluation. However, to validate that the study toolkit actually worked to enable our full study and to show the reusability of our utilities, we prepared a way for partial reproduction.

The reproduction being partial means: reproducing our study on a smaller dataset but covering the complete study process (i.e., from prepared benchmarks to final results).

Specifically, the smaller dataset includes four apps, as randomly chosen from our entire benchmark suite. This benchmark subset is located at **samples/**.

We provide three ways of partial reproduction/replication, providing three options for the reviewers.

1. Quick reproduction

To reproduce, do

```
cd /home/hcai/issta-ae/code
bash partial-reproduce.sh
```

This will run our data analysis code on pre-generated installation logs (for 2140 apps of year 2010 against API Level 19) and app traces (for the four apps chosen against API level 21).

The expected result is as follows:

(note: as in other expected results, this shows how many apps were not successfully installed, and out of the apps for which the installation failed, the category distribution of failure error messages; and similar results on the run-time compatibility study --- how many failed at run-time due to incompatibilities and the category distribution of error messages for failed apps. It was from these error messages that we figured out the symptoms and corresponding causes of installation-/run-time compatibility issues)

```

[hcai@hcai-d1580:~/issta-ae-fixed/code$ bash partial-reproduce.sh
===== Installation-time compatibility study =====
In 2140 apps :      successful: 2095      failed: 45

Reviewer expertise
< 1 >: Effect: Failure [INSTALL_FAILED_MISSING_SHARED_LIBRARY]
      -- SDK 3 : 16
      -- SDK 4 : 9
      -- SDK 8 : 9
      -- SDK 21 : 2
      -- SDK 23 : 1

Review #4C Updated 23 May 2019 11:38:10pm EDT
< 2 >: Effect: Failure [INSTALL_FAILED_CONFLICTING_PROVIDER]
      -- SDK 3 : 4
      -- SDK 4 : 1
      -- SDK 8 : 2

Basic functionality
3. Good
< 3 >: Effect: Failure [INSTALL_FAILED_ALREADY_EXISTS]
      -- SDK 3 : 1

Comments on basic functionality
===== Run-time compatibility study =====
Total = 4 Successful = 3 Failed = 1
< 1 >: Effect: Null pointer
      -- SDK 1 : 1

```

how does this result relate to the paper results?

For each study, the first line gives the overall statistics from which failure rates can be easily derived (i.e., the IIR and RIR mentioned in paper); These numbers are plotted in the Figures 2 and 5 of the paper.

Then, the rest of the output gives the distribution of failure cases over possible effects (incompatibility symptoms) and possible target (if not available, min) SDK versions.

For example, out of the 2140 apps of year 2010, 45 apps failed to be installed, exhibiting three kinds of symptoms (e.g., `INSTALL_FAILED_MISSING_SHARED_LIBRARY`); among the 7 apps that failed with the `INSTALL_FAILED_CONFLICTING_PROVIDER` effect, 4 apps specified SDK version 3, 1 app specified SDK version 4, and 2 apps specified SDK version 8. The format of the run-time study results is similar (e.g., in this case, 1 app failed exhibiting the effect of Null pointer). These distribution results are what we showed in the Figures 4 and 7 of the paper.

2. Quick demo

To run the demo, do

```
cd /home/hcai/issta-ae/code
```

```
bash demo-quick.sh
```

This demonstrates how our tools would be reused on a new dataset (i.e., **samples/** here as an example), but for speed, we have generated the installation logs and traces beforehand (only for the four sample apps).

The expected result is as follows:

```
[hcai@hcai-d1580:~/issta-ae-fixed/code$ bash demo-quick.sh
===== Installation-time compatibility study =====
res_run.txt
In 4 apps :      successful: 4      failed: 0
runAndroZooApks_monkey.sh
===== Run-time compatibility study =====
sampleInstallationlog.txt
Total = 4 Successful = 3 Failed = 1
samples
< 1 >: setEffect: Null pointer
-- SDK 1: 1
signature.sh 1
```

The format of this result is the same as explained above for the partial reproduction.

3. Full demo

To run the demo, do

```
cd /home/hcai/issta-ae/code
bash demo.sh
```

This demonstrates how our tools would be reused on a new dataset (e.g., **samples/**) from scratch, starting from the app APKs all the way to the study results. (Note, even for the very small dataset, this full demo may take 20 minutes or longer---please see 'Known issues' below for the explanation; also if this demo is executed, then the pre-generated data may be cleaned, thus the quick demo may not be executed anymore before the full demo is completed.) Please also note that, for speed, the default profiling time was set to one minute per app, although in our study the time was five minutes. Customized time period (in the unit of minutes) can be passed as the second command line argument to *runAndroZooApks_monkey.sh*. (e.g., for running each app for five minutes, use *runAndroZooApks_monkey.sh samples 5* inside *demo.sh*).

The expected eventual result should be the same as the result from the quick demo.

For each of the three ways, results can be viewed from the screen, or inspected from the two files below that should be generated if the reproduction/demo executed successfully:

install_effect_sdk.txt (containing the installation-time study result)

benign_run_effect_sdk.txt (containing the run-time study result)

Please note that none of the results produced above would immediately match what is shown in the paper, because the results shown in paper are collective/aggregate (summary) statistics computed from the results demonstrated here for **all** benchmarks. The complete results that match what we presented in the paper can be found under the **data/** folder (mostly in the Excel files).

Section IV: Known issues

It is known that the Android emulator is best run on Intel-x86 architecture that has hypervisor support (kvm). However, the supervisor support is rarely available on a VM such as ours created using and running on Virtual Box.

As a makeshift, we chose to run the study in the VM on an Android AVD based on armeabi-v7a without gpu support. In consequent, the emulator runs much slower than running on a physical machine. In addition, our benchmarks may behave differently on an Intel machine (where we conducted the original study) than on the ARM machine (where we show the artifact through the VM). Thus, the performance issues and results variance can occur. However, this should only affect the partial result reproduction which involves app executions. Producing the final results from raw study results (by running `/home/hcai/issta-ae/partial-reproduce.sh`) should not be affected.

Appendix: use the artifact on a different machine

1. Requirements for experimental environment

Our artifacts have been tested with the following environment settings and tools/libraries (i.e., the configurations on a clean machine):

- Ubuntu 16.04 LTS
- Java 1.8.0_131 (java-1.8.0-openjdk-adm64; including the jarsigner tool)
- Python 2.7 (and the xlwt library)
- Tree 1.7.0 (the tree tool for browsing directories)
- Android SDK (with API 19, including tools liked adb, emulator, monkey, etc.)
- Android AVD (Nexus-One-10 with 1G SD card, 200 RAM, and intel-x86 64 system image)
- expect (year 1994 version)
- aapt (year 2014 version)
- Apktool v2.0.2
- ant v1.9.6
- vim 7.4 (optional, for editing scripts and viewing textual results)
- zipalign ("sudo apt install zipalign")
- All the libraries as listed in `/home/hcai/lib`s
 - For users who just want to use artifact.zip independently (instead of referring to the VM image), these libraries can be downloaded here: <http://chapering.github.io/droidfax.zip>

If using the artifact independently, beyond successfully installing the above tools/libraries, users will also need to set the `ROOT` variable in `cgInstr.sh` to the directory where they would put the **libs** downloaded as described above before the full demo can function properly.

For setting up the environment as reflected in the requirements above, users may simply refer to the installation guide for the Droidfax toolkit --- most of the utilities used for this study (in particular, those for installing/running an app on an emulator) were part of that toolkit.

The Droidfax project website is here: <http://chapering.github.io/droidfax/>, where detailed installation guide is provided, along with a demo video.

2. Role/functionality/usage of toolkit

To facilitate reusing our toolkit, the following text give the usage summary of our code artifact items (under the **code/** directory), describing the functionality of each program/tool. The first part is dedicated for the code used for the installation-time compatibility study, while the second for the code used for the run-time compatibility study. The descriptions assume the tools are used against new data from users, not bounded to the experiments in the paper.

For installation-time compatibility study:

- **installAndroZooApks.sh** apk_dir [avdname] [port]
 - role: This tool tries to install each APK under the given directory of APKs (apk_dir) and then uninstall it on the given device (avd) at the given port (port)--default avd is "Nexus-One-10" and default port is 5572, and then collects the log (whatever the installation/uninstallation steps output, including whether the installation succeeds or if not any error code/messages produced); the log is dumped to stdout by default
 - command line arguments:
 - apk_dir: the directory of APKs to be studied
 - avdname: the name of the Android device to install to (uninstall from)
 - port: the port to run the device at
- **classify_failed_installation_effect_sdk.py** installation_log_file
 - role: This tool parses the installation log specified by the argument to compute statistics on failure rate and distribution of failure effects (according to the error code/messages produced); it outputs the results to stdout by default.
 - command line arguments:
 - installation_log_file: the log file holding all the installation logs produced by the installation process (via **installAndroZooApks.sh**)

For run-time compatibility study:

- **instrSamples.sh** apk_dir
 - role: This tool instruments each APK in the given directory (apk_dir) and produces the instrumented APK under the *cg.instrumented* directory that it creates if it did not exist; the instrumentation is for inserting probes that monitor method calls during app executions, which are later used to determine whether the apps execute normally (in the sense of being compatible or not, rather than functionally normal or not) or not.
 - command line arguments:

- apk_dir: the directory of APKs to be studied
- **runAndroZooApks_monkey.sh** apk_dir [timeout] [avdname] [port]
 - role: This tool tries to install each APK under the given directory of APKs (apk_dir), launches it and exercises it using automated test inputs generated by the Monkey tool as part of Android SDK for the given period of time (timeout) and then uninstall it, all on the given device (avd) at the given port (port)--- default avd is "Nexus-One-10" and default port is 5572; after the execution, it collects the execution trace and stores it into \${apk-name}.logcat under the *androZooLogs/\${apk_dir}* directory---the name includes 'androZoo' but this does not mean the APKs need to be from the AndroZoo database.
 - command line arguments:
 - apk_dir: the directory of APKs to be studied
 - avdname: the name of the Android device used for the execution
 - port: the port to run the device at
- **getanysdkall.sh** apk_dir
 - role: retrieves the targetSdkVersion (if not available, falling to retrieve minSdkVersion instead) from each APK in the given directory (apk_dir), and produces either the SDK version or error message per APK, to a file 'res.txt'
 - command line arguments:
 - apk_dir: the directory of APKs to be studied
- **checkallsdk.py** apk_dir
 - role: retrieves the 'res.txt' file produced by getanysdkall.sh above and normalizes the SDK version (to a number in [1,27]) for each APK for which the version was retrieved successfully --- the normalization simply means (1) only keeping the results for APKs that have the version information, and (2) converts the hexadecimal number in app manifest that specifies the version to the corresponding decimal number. The result is produced to stdout by default.
 - command line arguments:
 - apk_dir: the directory of APKs to be studied
 - Note: getanysdkall.sh and checkallsdk.py work together to simply record the SDK version of each APK, which is later used to compute the distribution of run-time incompatibility-induced failure effects over these SDK versions specified in the APKs.
- **classify_failed_running_effect_minsdk.py** run-time-log-dir
 - role: This tool parses each run-time log (i.e., the execution trace corresponding to each APK) to determine if the APK was running normally, or identify the failure effect (based on the error message or code in the log); then, using the output produced by checkallsdk.py above, it computes the failure rate and distribution of failure effect over various SDK versions
 - command line arguments:
 - run-time-log-dir: the directory that holds the run-time log per APK (i.e., the *androZooLogs/\${apk_dir}* directory mentioned in the usage of runAndroZooApks_monkey.sh above)

Common utilities: (used by both studies)

- **apkinstall** apk-file-name [device-id]

- role: install the given APK to the specified device (default: emulator-5554)
- **apkuninstall** apk-file-name [device-id]
 - role: uninstall the given APK to the specified device (default: emulator-5554)
- **cglInstr.sh** apk-file-name
 - role: instrument the given APK for tracing (as required for our run-time compatibility study)
- **getanysdk.sh** apk-file-name
 - role: retrieves the targetSdkVersion (if not available, falling to retrieve minSdkVersion instead) from the given APK
- **getminsdk.sh** apk-file-name
 - role: retrieves the minSdkVersion of the given APK
- **getmaxsdk.sh** apk-file-name
 - role: retrieves the maxSdkVersion of the given APK
- **gettargetsdk.sh** apk-file-name
 - role: retrieves the targetSdkVersion of the given APK
- **getpackage.sh** apk-file-name
 - role: retrieves the package name of the given APK
- **setupEmu.sh** avdname [port]
 - role: launches the android device specified by the avdname argument, at the port given (default: 5554)
- **signandalign.sh** apk-file-name
 - role: sign the given APK (used in cglInstr.sh to sign the instrumented APK; without being signed, the APK would not be able to be installed to an Android device)

3. Run entire study on new benchmarks

To run through the entire study (including installation- and run-time compatibility studies) on a new benchmark suite, simply put the APK files under **samples/** directory, and then run the complete demo via *demo.sh*.

Inside *demo.sh*, you may find the study is clearly separated into two parts, the first for installation-time study and the second for run-time study. You can run these two parts separately by simply executing the corresponding scripts that are used for each part.

Moreover, as you may notice, each study contains two main steps: (1) collect log/trace, and (2) compute study report. Users have the flexibility to execute these two steps separately too. Generally, the last script for each study represents Step (2) and all other preceding scripts represent Step (1).

For instance, for the run-time study, the following scripts are for Step (1):

```
bash ./instrSamples.sh samples
bash ./runAndroZooApks_monkey.sh samples
bash ./getanysdkall.sh samples
python ./checkallsdk.py samples > res_run.txt
```

While the last script corresponds to Step (2) of the run-time study:

```
python ./classify_failed_running_effect_minsdk.py androZooLogs/samples
```